

[Description](#) | [Accepted](#) x | [Editorial](#) | [Solutions](#) | [Submissions](#)

[All Submissions](#)

Accepted 488 / 488 testcases passed

 **yousab21** submitted at Apr 28, 2026 19:54

Runtime

0 ms | Beats 100.00% 

Memory

9.99 MB | Beats 53.11% 



Runtime (ms)	Percentage
0	~65%
1	~5%
2	~5%
3	~15%
4	~5%
5	~2%
6	~2%

Code | C++

```
1 class Solution {
2 public:
3     bool isPalindrome(string s) {
4         int left = 0;
5         int right = s.size() - 1 ;
6
7
8         while (left < right) {
```

 View more

More challenges







Write your notes here

Select related tags 0/5

Code

C++ v Auto

```
4 class Solution {
3 public:
2     bool isPalindrome(string s) {
1         int left = 0;
5         int right = s.size() - 1 ;

1
2
3         while (left < right) {
4
5             //this part is to skip any characters that are not characters (inculding spaces)
6             // std::isalnum() is a carried over functionf from C it returnts true
7             // if aa charachter is a alphapet character or a number digit and false if it is ant other ascii value
8             while (left < right && !isalnum(s[left])){
9                 left++;
10            }
11            while (left < right && !isalnum(s[right])){
12                right--;
13            }
14
15            // i check the lower case characters at each of the two pointer and if at any
16            // point they dont match wa cam say tehe string is not a palindrome and return false
17            if (tolower(s[left]) != tolower(s[right])){
18                return false;
19            }
20
21            // increment the pointers together
22            left++;
23            right--;
24        }
25
26        // if the string pass all the checks without being flaged it is a palindrome
27        return true;
28    }
29 };
```

Saved

Testcase

 Test Result

Ln 5, Col 32

Problem List

DescriptionAcceptedEditorialSolutionsSubmissions

All Submissions

Accepted24 / 24 testcases passed

yousab21 submitted at Apr 28, 2026 20:06

Analysis

Solution

Runtime

0 ms | Beats 100.00%

Memory

19.40 MB | Beats 94.43%

Runtime	1ms	2ms	3ms	4ms
0 ms	1ms	2ms	3ms	4ms

Code | C++

```
1 class Solution {
2 public:
3     vector<int> twoSum(vector<int>& numbers, int target) {
4         //here we can take advantage that the array is sorted and use a 2 po
5         //first we intialize the 2 pointer in the opisite sides of the array
6         // that makes it that left is on the smallest element and right on t
7         int left = 0;
8         int right = numbers.size() - 1 ;
9     }
10 }
```

View more

More challenges

653. Two Sum IV - Input is a BST

1099. Two Sum Less Than K

Write your notes here

Select related tags0/5

✓

Code

C++Auto

```
36 class Solution {
37 public:
38     vector<int> twoSum(vector<int>& numbers, int target) {
39         //here we can take advantage that the array is sorted and use a 2 pointer solution
40         //first we intialize the 2 pointer in the opisite sides of the array
41         // that makes it that left is on the smallest element and right on the highestst
42         int left = 0;
43         int right = numbers.size() - 1 ;
44
45         int sum ;
46         vector<int> result;
47
48         while(left < right){
49             // we calculate the sum of elements at the pointers locations and check
50             sum = numbers[left] + numbers[right];
51             //here we have 3 possibilties
52
53             // 1- the sum = the target :
54             // then we are done. we set the result and break
55             if (sum == target){
56                 result = {left+1 , right +1};
57                 break;
58             }
59
60             //2- the sum is smaller than the target
61             //then we move the left pointer right, that increases the sum scince the array is sorted
62             else if(sum < target){
63                 left++;
64             }
65
66             //3- the sum is greater than the target
67             //then we move the right pointer left, that dencreases the sum scince the array is sorted
68             else if (sum > target){
69                 right --;
70             }
71
72             // we keep doing this until we either hit the target or the pointer cross at which point there is not anser
73         }
74         return result;
75     }
76 };
77 }
```

Saved

Ln 37, Col 120

Testcase

Test Result

Two Pointers

DescriptionAccepted xEditorialSolutionsSubmissions

All Submissions

Accepted62 / 62 testcases passed

AnalysisSolution

yousab21 submitted at Apr 28, 2026 21:30

Runtime

18 ms | Beats 5.07%

Memory

50.46 MB | Beats 5.08%

CodeC++

```
1 class Solution {
2 public:
3 //to make the code more readable i split it into those helper functions
4
5 void findNextWord(string& s,int& ptr){
6     //this function takes a pointer at any point in any word and finds t
7     //it does this by moving the pointer left until it finds a space
8     //basically finding the index that begins a new word
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

View more

More challenges

186. Reverse Words in a String II

Write your notes here

Select related tags0/5

✓

Code

C++Auto

```
65 class Solution {
66 public:
67 //to make the code more readable i split it into those helper functions
68
69 void findNextWord(string& s,int& ptr){
70     //this function takes a pointer at any point in any word and finds the begging of it
71     //it does this by moving the pointer left until it finds a space
72     //basically finding the index that begins a new word
73     while(ptr > 0){
74         if(s[ptr] != ' ' && s[ptr - 1] == ' '){
75             break;
76         }
77         ptr--;
78     }
79     // it leaves the pointer at the first char of the word
80 }
81
82 string getWord(string& s, int& ptr){
83     //this function takes runs after we get the index of the new word
84     //and it returns a string of all the characters in the word until it hits a space
85     //or the array ends in case the words was the last one in the string
86     int current = ptr;
87     string word = "";
88     while(s[current] != ' ' && current < s.size() ){
89         word = word + s[current];
90         current++;
91     }
92     return word;
93 }
94
95 void skipExtraSpaces(string& s,int& ptr){
96     //because findNextWord() relays at having a pointer on an actual word
97     //and because white spaces are random all over the the string
98     // i made this function that just moves the pointer to igonre all white spaces
99     // and set it at the first word of it hits
100    while(s[ptr] == ' ' && ptr > 0){
101        ptr--;
102    }
103 }
104
105 string reverseWords(string s) {
106     string newString = "";
107
108     //i begin with the pointer at the last char on the string
109     int ptr = s.size() -1;
110     while(ptr >= 0 ){
111         // and i call my functions in this order
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
```

SavedLn 66, Col 26

TestcaseTest Result

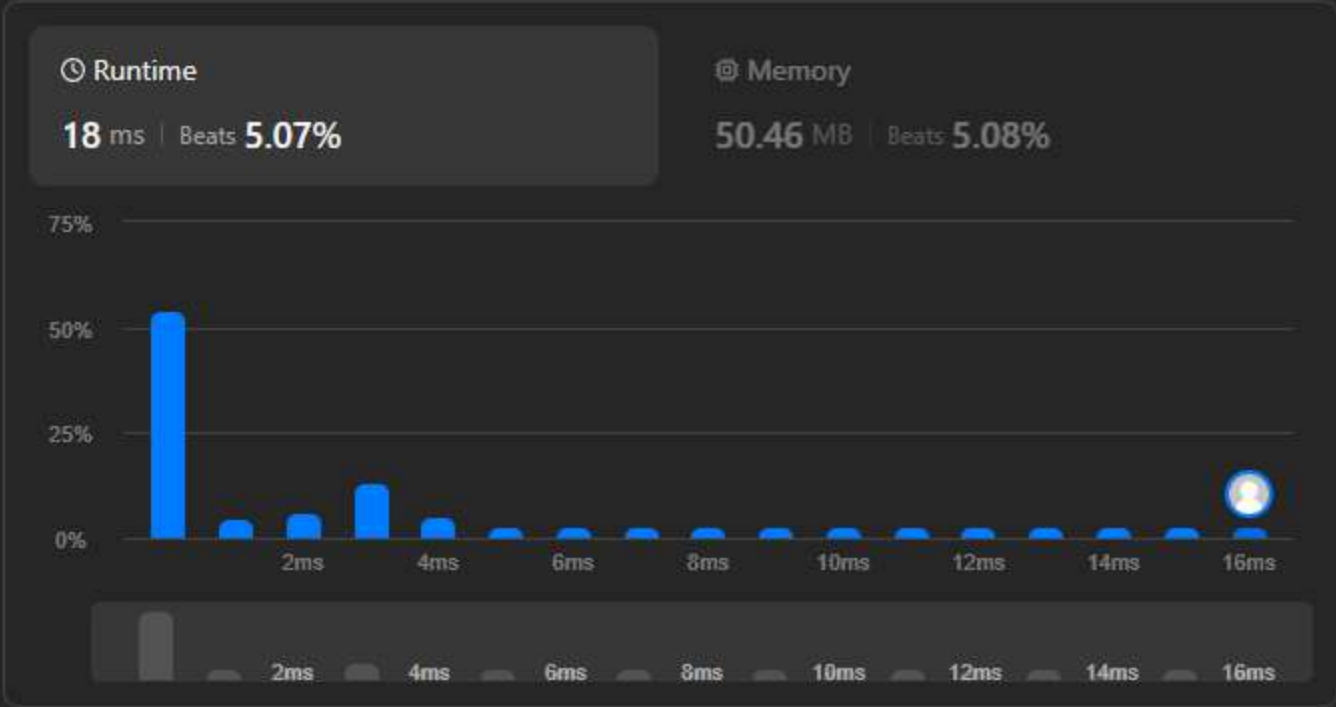
AcceptedRuntime: 0 ms

Accepted62 / 62 testcases passed

Analysis

Solution

yousab21 submitted at Apr 28, 2026 21:30



Code | C++

```
1 class Solution {
2 public:
3 //to make the code more readable i split it into those helper functions
4
5 void findNextWord(string& s,int& ptr){
6     //this function takes a pointer at any point in any word and finds t
7     //it does this by moving the pointer left until it finds a space
8     //basically finding the index that begins a new word
9
10    }
11
12    }
13
14    }
15
16    }
17
18    }
19
20    }
21
22    }
23
24    }
25
26    }
27
28    }
29
30    }
31
32    }
33
34    }
35
36    }
37
38    }
39
40    }
41
42    }
43
44    }
45
46    }
47
48    }
49
50    }
51
52    }
53
54    }
55
56    }
57
58    }
59
60    }
61
62    }
63
64    }
65
66    }
67
68    }
69
70    }
71
72    }
73
74    }
75
76    }
77
78    }
79
80    }
81
82    }
83
84    }
85
86    }
87
88    }
89
90    }
91
92    }
93
94    }
95
96    }
97
98    }
99
100   }
```

View more

- More challenges
- 186. Reverse Words in a String II

Write your notes here

Select related tags0/5



```
40     current++;
39     }
38     return word;
37 }
36
35 void skipExtraSpaces(string& s,int& ptr){
34     //because findNextWord() relays at having a pointer on an actual word
33     //and because white spaces are random all over the the string
32     // i madde this function that just moves the pointer to igonre all white spaces
31     // and set it at the first word of it hits
30     while(s[ptr] == ' ' && ptr > 0){
29         ptr--;
28     }
27 }
26
25 string reverseWords(string s) {
24     string newString = "";
23
22     //i begin with the pointer at the last char on the string
21     int ptr = s.size() -1;
20     while(ptr >= 0 ){
19         // and i call my functions in this order
18         skipExtraSpaces(s , ptr);
17         findNextWord(s, ptr);
16         // this sets up the pointer to read the next word so i do that
15         // and i concatenate it with my result string
14         newString = newString + getWord(s , ptr) + " ";
13         // the way i do this adds an extra white space at the end but deal with it later
12         ptr--;
11     }
10
9     //words that have white spaces in the begging have aside effect of those whitespaces
8     //wrapping up in the end of the result sting so i loop at the stift from the end and
7     //remove any trailing whitespaces (that also cleans up the extra space i mentioned before)
6     int i = newString.size() - 1;
5     while(newString[i] == ' '){
4         newString.pop_back();
3         i--;
2     }
1
66     return newString;
1
2 };
```

Accepted

Runtime: 0 ms

Description Accepted x Editorial Solutions Submissions

All Submissions

Accepted 40 / 40 testcases passed

yousab21 submitted at Apr 29, 2026 19:38

Analysis

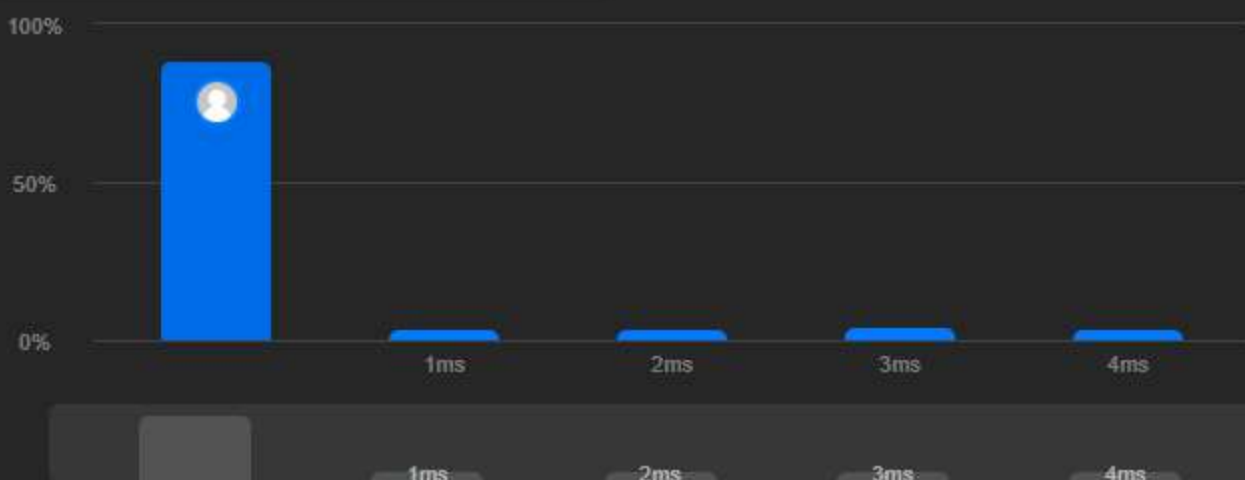
Solution

Runtime

0 ms Beats 100.00%

Memory

31.01 MB Beats 5.45%



Code C++

```
1 class Solution {
2 public:
3
4     //insted of rotating the array one rotations at a time i made this hlper
5     //it takes a range from one array and adds it to another i will explain
6     void addRange(vector<int>& distination, vector<int>& target, int start ,
7         for(int i = start ; i <= end ; i++){
8         distination.push_back(target[i]);
```

View more

More challenges

- 186. Reverse Words in a String II
- 2607. Make K-Subarray Sums Equal
- 3400. Maximum Number of Matching Indices After Right Shifts

Write your notes here

Select related tags

0/5

Code

C++ Auto

```
33 class Solution {
32 public:
31
30     //insted of rotating the array one rotations at a time i made this hlper function
29     //it takes a range from one array and adds it to another i will explain how i will use it in a second
28     void addRange(vector<int>& distination, vector<int>& target, int start , int end){
27         for(int i = start ; i <= end ; i++){
26             distination.push_back(target[i]);
25         }
24     }
23
22     void rotate(vector<int>& nums, int k) {
21         //the old get rid of unnessery rotations trick
20         k = k % nums.size();
19
18         // i made this to act as my result array
17         vector<int> nums2;
16
15         // here i copy the last k number of elemts as the first elements in
14         //th result array this is the first part of the rotation problem
13         int start = nums.size() - k;
12         int end = nums.size() - 1;
11         addRange(nums2, nums, start, end);
10
9         //now i copy the remaining elemtns from the original array to the end
8         // of the result array this completes the idea of rotation by k
7         start = 0;
6         end = nums.size() - k - 1; ;
5         addRange(nums2, nums, start, end);
4
3         // since we dont return any thing i make the original array refrence the result
2         nums = nums2;
1     }
34 };
```

Saved

Ln 34, Col 3

Testcase Test Result

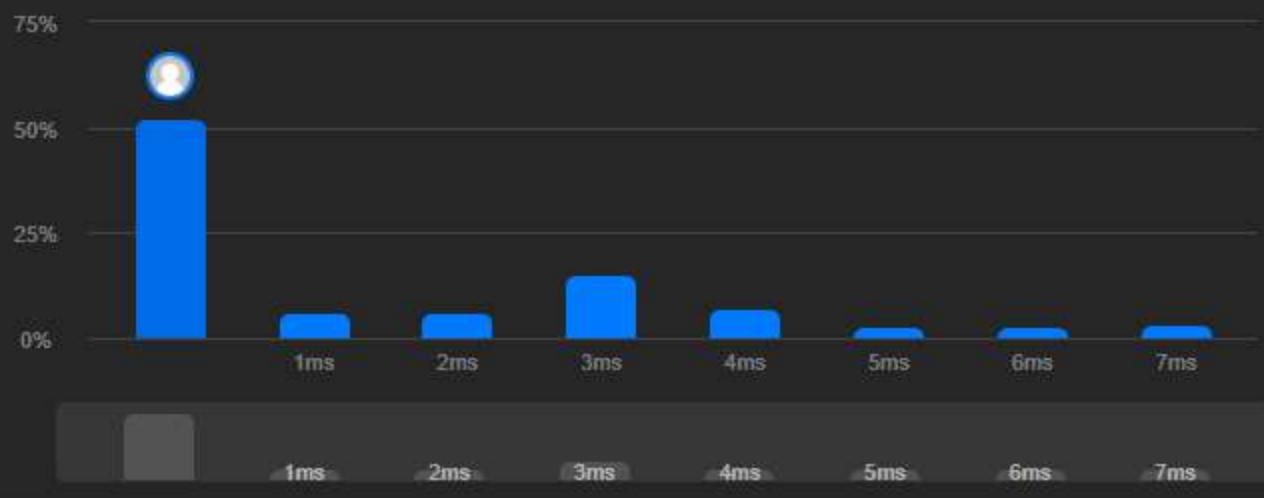
Description Accepted x Editorial Solutions Submissions

All Submissions

Accepted 480 / 480 testcases passed
yousab21 submitted at Apr 29, 2026 19:53

Analysis Solution

Runtime 0 ms | Beats 100.00%
Memory 10.17 MB | Beats 91.18%



Code C++

```
1 class Solution {
2 public:
3     //this reads itself
4     bool isAVowel(char c){
5         switch(c){
6             case 'a':return true;
7             case 'A':return true;
8             case 'e':return true;
```

View more

More challenges

- 344. Reverse String
- 1119. Remove Vowels from a String
- 2810. Faulty Keyboard

Write your notes here

Select related tags 0/5



Code

C++ Auto

```
28 class Solution {
29 public:
30     //this reads itself
31     bool isAVowel(char c){
32         switch(c){
33             case 'a':return true;
34             case 'A':return true;
35             case 'e':return true;
36             case 'E':return true;
37             case 'i':return true;
38             case 'I':return true;
39             case 'o':return true;
40             case 'O':return true;
41             case 'u':return true;
42             case 'U':return true;
43         }
44     }
45     return false;
46 }
47
48 string reverseVowels(string s) {
49     //here is use 2 pointer from the different ends of the string
50     //to swap the vowels
51     int l = 0;
52     int r = s.size() - 1;
53
54     while(l < r){
55         //first i move the pointers inwards until they both stand on vowels skipping any non vowels
56         // then i swap those vowels
57         // i check for l<r because if i dont they can cross each other and re-swap vowels that have already been swapped
58         while(l < r && !isAVowel(s[l])) {l++;}
59         while(l < r && !isAVowel(s[r])) {r--;}
60
61         swap(s[l], s[r]);
62
63         l++;
64         r--;
65     }
66     return s;
67 }
68 };
```

Saved

Ln 29, Col 40

Testcase Test Result

[Question](#) | [Solution](#) | [Submissions](#) | [Discuss \(9\)](#)

[← All Submissions](#)

Accepted 21 / 21 test cases

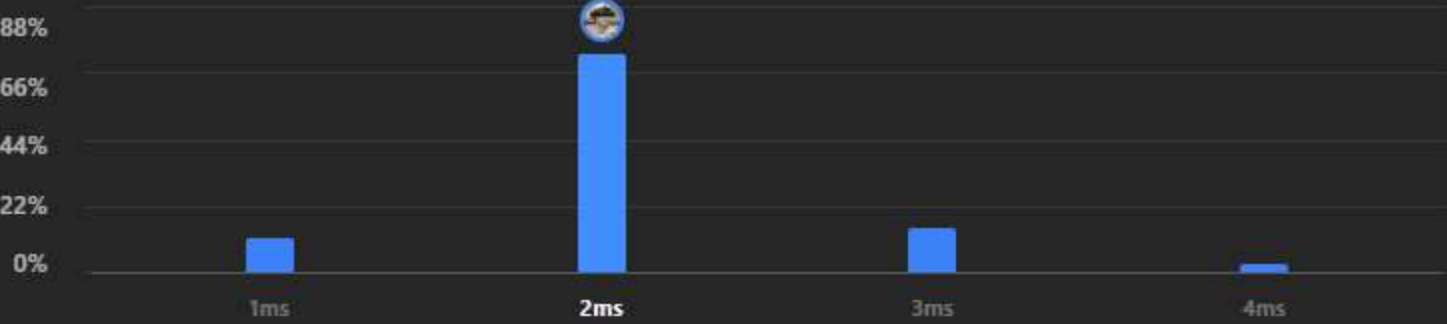
Memory: 104.7 MB • Time: 2ms • Submitted at: 04/29/2026 20:24 •

Runtime

2ms | Beats 88.98%

Memory

104.7 MB | Beats 96.61%



[Analyze Complexity](#)

Code | C++

[Visualize code](#) [Share Solution](#)

```
//this is just merge from merge sort but easier
class Solution {
public:
    string mergeAlternately(string word1, string word2) {
        int i = 0, j = 0;
        string word3;
        while(i < word1.size() && j < word2.size()){
            // i just take one char from each word until one word is done
            word3.push_back(word1[i]);
            word3.push_back(word2[j]);
            i++;
            j++;
        }

        //then i copy the rest of the other word
        while(i < word1.size()){
            word3.push_back(word1[i]);
        }
    }
};
```

C++ i • Auto

[Solution 1](#) +

[NeetBot](#) | [Hint](#) | | Ln 15, Col 49

```
1 //this is just merge from merge sort but easier
2 class Solution {
3 public:
4     string mergeAlternately(string word1, string word2) {
5         int i = 0, j = 0;
6         string word3;
7         while(i < word1.size() && j < word2.size()){
8             // i just take one char from each word until one word is done
9             word3.push_back(word1[i]);
10            word3.push_back(word2[j]);
11            i++;
12            j++;
13        }
14
15        //then i copy the rest of the other word
16        while(i < word1.size()){
17            word3.push_back(word1[i]);
18            i++;
19        }
20
21        while(j < word2.size()){
22            word3.push_back(word2[j]);
23            j++;
24        }
25
26        return word3;
27    }
28 };
```

Console

[Run](#) [Submit](#)

← All Submissions

Accepted 28 / 28 test cases

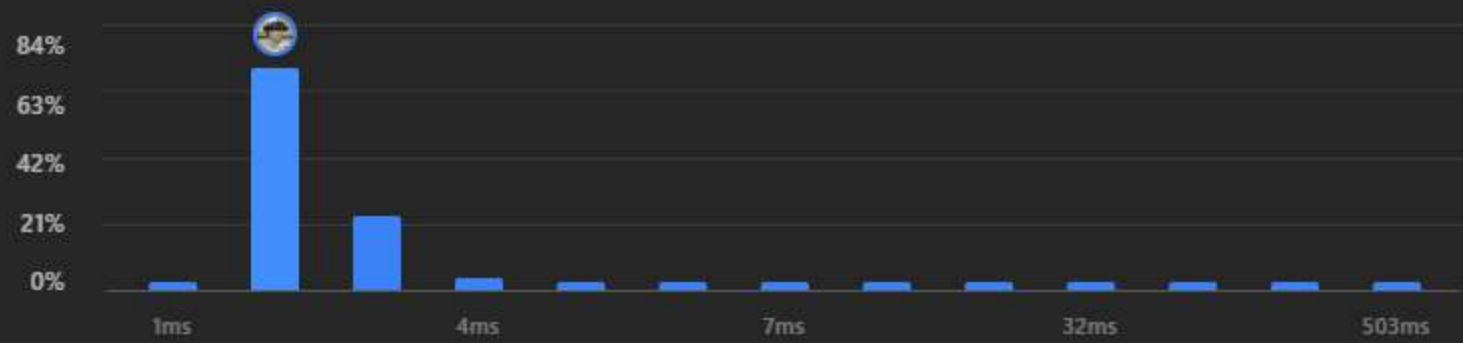
Memory: 105 MB · Time: 2ms · Submitted at: 04/29/2026 20:57 ·

Runtime

2ms | Beats 97.34%

Memory

105.0 MB | Beats 85.48%



Analyze Complexity

Code | C++

Visualize code

Share Solution

```
class Solution {
public:
    //this took me some time to figure out the non brute force solution

    //this is just a helper funciotn to calculate the water between two bars
    int getWater(vector<int>& heights, int& l , int& r){
        //the water between the two bars is limited by 2 things
        //1- the shortest bar (because if the water was any heighr ir will spill)
        //2- the width of the contaner (the space between the bars)
        return min(heights[l],heights[r]) * (r - l);
    }

    int maxArea(vector<int>& heights) {
        //i start by using 2 pointers on differewnt sides of the array
        int l = 0;
        int r = heights.size()-1;
        int water = 0;
```

Solution 1 +

NeetBot | Hint | Ln 33, Col 18

```
1 class Solution {
2 public:
3     //this took me some time to figure out the non brute force solution
4
5     //this is just a helper funciotn to calculate the water between two bars
6     int getWater(vector<int>& heights, int& l , int& r){
7         //the water between the two bars is limited by 2 things
8         //1- the shortest bar (because if the water was any heighr ir will spill)
9         //2- the width of the contaner (the space between the bars)
10        return min(heights[l],heights[r]) * (r - l);
11    }
12
13    int maxArea(vector<int>& heights) {
14        //i start by using 2 pointers on differewnt sides of the array
15        int l = 0;
16        int r = heights.size()-1;
17        int water = 0;
18
19        while(l < r){
20            // every iteration i calculate the amount of water
21            // and compare it to the max at this point
22            water = max(water, getWater(heights, l , r));
23
24            //here is the intrtesting part :
25            //since the water is bound by shorter of the two bars
26            // moving the pointer at the heighr bar will only lower the "potintial" water i can get
27            //so i only move the pointer at the shorter bar as this is the only way water "may" increase
28            //in hope finding a bigger bar would compensate for the lost width
29            if(heights[l] < heights[r]){
30                l++;
31            }
32
33            else{
34                r--;
35            }
36        }
37        return water;
38    }
39 };
40
```

Console

Run

Submit